



Escuela de Ingeniería en Computadores

CE1106 – Paradigmas de Programación

Tarea #3

Prolog - Paradigma Lógico

Integrantes:

Andrés Enrique Blanco Coto - 2022108841

Gabriel Bolaños Barboza – 2022327511

Fabiana María Moreno Castrillo - 2023153700

Profesor: Luis Alonso Barboza Artavia

Fecha de entrega: 03 de mayo del 2026

Tabla de contenido

Reglas implementadas	4
Reglas de Análisis Sintáctico (Parser DCG)	4
“interpretar_texto/3” y “interpretar_texto/4”	4
“oracion//3”	4
“sintagma_verbal//3”	4
Reglas del Motor de Inferencia	4
“puntaje_lista/4”	4
“puntaje_positivo/3” y “puntaje_negativo/3”	4
“ajustar_por_nivel/3”	4
“recomendar/3”	4
Estructuras de datos desarrolladas	4
Listas de Evidencias (Términos Compuestos)	5
Listas de Tokens (Cadenas a Átomos)	5
Listas de Pares (Puntaje-Profesión)	5
Base de Conocimientos (Hechos Atómicos)	5
Algoritmos desarrollados	5
Algoritmo de Análisis Sintáctico (Parser en BNF.pl)	5
Tokenización y Normalización	5
Análisis DCG (Definite Clause Grammars)	5
Extracción de Semántica	5
Regla de Rechazo	6
Motor de Inferencia y Cálculo de Puntajes (Logic.pl)	6
Recolección	6
Evaluación de Puntajes	6
Ajuste por Nivel de Intensidad	6
Selección del Máximo	6
Problemas sin solución	6
Errores ortográficos no previstos	6
Sarcasmo y frases complicadas	6
Diccionario de palabras estático	7
Plan de Actividades	7

Problemas encontrados.....	8
Problema 1: Rechazar el “Sí/No” directo.....	8
Problema 2: Recursividad duplicada en el ciclo de preguntas	8
Problema 3: Desfase de preguntas por confusión	9
Conclusiones y Recomendaciones del proyecto	9
Conclusiones	9
Recomendaciones	9
Bibliografía	10

Reglas implementadas

El sistema utiliza un conjunto de reglas lógicas divididas en dos grandes áreas: el análisis del lenguaje natural y el motor de inferencia experto. A continuación, se describen las reglas más importantes:

Reglas de Análisis Sintáctico (Parser DCG)

`"interpretar_texto/3"` y `"interpretar_texto/4"`

Son el punto de entrada para procesar la entrada del usuario. Se encargan de llamar a la regla de tokenización y luego aplican la gramática libre de contexto.

`"oracion//3"`

Es la regla principal de la gramática. Define que una oración válida puede contener relleno inicial, una oración base (que combina sintagmas nominales y verbales, o respuestas cortas) y relleno final.

`"sintagma_verbal//3"`

Reglas que mapean verbos y estados a una intención (positiva o negativa) y a un nivel de intensidad. Por ejemplo, relacionan "amo" con una intención positiva y "odio" con una negativa.

Reglas del Motor de Inferencia

`"puntaje_lista/4"`

Es una regla recursiva fundamental. Recorre la lista de evidencias dadas por el usuario, evalúa el tipo de evidencia (positiva o negativa), calcula los puntos correspondientes llamando a `"puntaje_positivo"` o `"puntaje_negativo"`, ajusta dichos puntos según el nivel de intensidad, y los acumula.

`"puntaje_positivo/3"` y `"puntaje_negativo/3"`

Asignan los pesos numéricos. Por ejemplo, una afinidad directa otorga +2 puntos y una fortaleza +1. En caso de intenciones negativas, una antagonía resta 3 puntos.

`"ajustar_por_nivel/3"`

Regla multiplicadora que modifica el puntaje base. Si el nivel es "alto" duplica los puntos; si es "bajo" los divide a la mitad (división entera), y si es "medio" los mantiene igual.

`"recomendar/3"`

Utiliza el predicado integrado `"findall"` para generar todos los puntajes posibles de las 10 profesiones y luego llama a `"mejor_resultado/2"` para recorrer esa lista y encontrar la profesión con el mayor puntaje.

Estructuras de datos desarrolladas

Debido a la naturaleza del paradigma lógico, el proyecto utiliza listas y términos compuestos (predicados) como sus principales estructuras de datos en lugar de variables tradicionales:

Listas de Evidencias (Términos Compuestos)

Las respuestas del usuario se acumulan en una lista dinámica. Cada elemento de esta lista es un término compuesto (functor) que encapsula la intención, el atributo y opcionalmente el nivel semántico extraído. Ejemplo: [positiva(tecnología, alto), negativa(salud, bajo), positiva(resolver_problemas)].

Listas de Tokens (Cadenas a Átomos)

Para el procesamiento de lenguaje natural, las strings ingresadas por el usuario se separan y normalizan, pasando a minúsculas y sin tildes, convirtiéndose en listas de átomos. Esta es la estructura base que consume el DCG. Ejemplo: [yo, amo, las, matematicas].

Listas de Pares (Puntaje-Profesión)

Al momento de calcular la recomendación final, se utiliza una lista que agrupa resultados en pares de la forma "Puntaje-Profesion". Esta estructura permite iterar de manera sencilla para comparar numéricamente cuál es el mayor. Ejemplo: [6-ingenieria_computadores, -3-medicina, 2-psicologia].

Base de Conocimientos (Hechos Atómicos)

Los datos estáticos del sistema experto se estructuran mediante relaciones (hechos) con aridad 1 y 2. Estas actúan como la base de datos que relaciona cada profesión con sus afinidades, fortalezas, antagonias y opuestos.

Algoritmos desarrollados

El sistema OrientadorCE se basa en dos algoritmos principales integrados: un analizador sintáctico (Parser) basado en gramáticas libres de contexto (BNF) y un motor de inferencia lógica para el cálculo de puntajes.

Algoritmo de Análisis Sintáctico (Parser en BNF.pl)

El algoritmo se encarga de traducir el lenguaje natural del usuario a una estructura semántica procesable:

Tokenización y Normalización

La entrada de texto del usuario se divide utilizando separadores (espacios y signos de puntuación). Luego, cada token se convierte a minúsculas y se le eliminan las tildes para estandarizar los átomos.

Análisis DCG (Definite Clause Grammars)

El algoritmo intenta hacer coincidir la lista de tokens con la regla principal "oracion/3". Evalúa si la estructura contiene un sintagma nominal, conectores, y un sintagma verbal, permitiendo variaciones en el orden y palabras de relleno.

Extracción de Semántica

Identifica la intención (positiva o negativa), el nivel de intensidad (alto, medio, bajo) y mapea las palabras clave a un atributo de la base de datos (ejemplo: matemáticas).

Regla de Rechazo

Si el usuario ingresa únicamente "sí" o "no", el algoritmo falla intencionalmente (false) para forzar al usuario a dar una respuesta articulada.

Motor de Inferencia y Cálculo de Puntajes (Logic.pl)

Una vez extraídas las evidencias con el Parser, el algoritmo lógico determina la profesión ideal:

Recolección

El sistema recopila todos los atributos que se pueden preguntar de la base de datos e itera sobre ellos, guardando las respuestas del usuario en una lista de evidencias.

Evaluación de Puntajes

Utiliza recursividad para recorrer la lista de evidencias contra la base de datos. Si la evidencia es positiva, suma puntos (+2 por afinidad directa, +1 por fortaleza). Si es negativa, resta puntos (-3 por antagonía, -2 si rechaza una afinidad).

Ajuste por Nivel de Intensidad

El algoritmo toma los puntos calculados y los ajusta, multiplica los puntos por 2 si la intensidad es "alta", los divide a la mitad si es "baja", y los mantiene intactos si es "media".

Selección del Máximo

Mediante el predicado "findall", el algoritmo recolecta el puntaje final de las 10 profesiones posibles. Finalmente, utiliza una función recursiva de comparación "mejor_resultado" para recorrer la lista y devolver la profesión con la puntuación más alta para ser recomendada al usuario.

Problemas sin solución

Aunque logramos que el proyecto cumpla con todos los requerimientos de la tarea, nos quedaron algunos detalles que por cuestiones de tiempo y complejidad no pudimos implementar:

Errores ortográficos no previstos

Le programamos al sistema que entienda algunos errores comunes que comete la gente (como escribir "facina" en lugar de "fascina"), pero si el usuario comete un error de tipeo muy extraño o inventa una palabra (tipo "tecnologíaa" o "matematicz"), el programa simplemente no reconoce el texto. No logramos implementar un algoritmo que calcule qué tan parecida es la palabra mal escrita a la correcta.

Sarcasmo y frases complicadas

El programa entiende súper bien oraciones directas, pero si el usuario le da muchas vueltas a la respuesta, usa dobles negaciones ("no es que no me guste...") o usa sarcasmo, el sistema de lenguaje natural se pierde y le pide al usuario que repita la respuesta.

Diccionario de palabras estático

Nos hubiera gustado que agregar nuevos sinónimos fuera más sencillo, pero actualmente muchas palabras clave están "quemadas" directamente en las reglas del archivo BNF.pl. Si en el futuro queremos que entienda más vocabulario, hay que ir a modificar el código fuente del parser en lugar de solo actualizar la base de datos.

Plan de Actividades

Descripción de la tarea	Responsable a cargo	Tiempo estimado	Fecha de entrega
Investigación y definición de las 10 profesiones, sus afinidades, fortalezas y antagonias en BD.pl.	Todos los integrantes	5 horas	23/04/2026
Diseño e implementación de las gramáticas libres de contexto (DCG) para el procesamiento de lenguaje natural en BNF.pl.	Gabriel Bolaños y Andrés Blanco	9 horas	26/04/2026
Desarrollo del motor de inferencia, cálculo de puntajes con niveles de intensidad e integración total en Logic.pl.	Andrés Blanco y Fabiana Moreno	10 horas	29/04/2026
Pruebas de integración del sistema, validación de casos de prueba y corrección de errores de recursividad.	Todos los integrantes	6 horas	01/05/2026
Elaboración de la documentación técnica, manual de usuario y descripción detallada de algoritmos.	Fabiana Moreno y Gabriel Bolaños	5 horas	02/05/2026
Creación de la presentación y diapositivas para la	Todos los integrantes	2 horas	03/05/2026

defensa del proyecto.			
-----------------------	--	--	--

Problemas encontrados

Problema 1: Rechazar el "Sí/No" directo

- Descripción detallada: En las instrucciones decía que el programa no podía aceptar un simple "sí" o "no" como respuesta, sino que el usuario tenía que escribir frases completas.
- Intentos de solución sin éxito: Primero tratamos de poner una validación rápida en el código principal para bloquear esas dos palabras cuando el usuario escribía. Pero no funcionó porque también nos bloqueaba respuestas que sí estaban buenas y empezaban con sí (por ejemplo: "sí, me gusta mucho").
- Soluciones encontradas: Nos dimos cuenta de que era mejor arreglarlo desde las reglas del lenguaje en BNF.pl. Lo que hicimos fue cambiar las reglas de "respuesta_corta" para obligar a que haya al menos una palabra extra después del sí o el no. Así, si alguien pone solo "sí", el sistema tira false a propósito y el archivo Logic.pl lo atrapa para decirle al usuario que no entendió e intente de nuevo.
- Recomendaciones / Conclusiones: Aprendimos que en Prolog es mejor meter todas las reglas de lo que el usuario puede o no escribir directamente en la gramática (el DCG), en lugar de estar haciendo validaciones feas en el código principal.
- Bibliografía consultada: Monferrer, T. E., Toledo, F., & Pacheco, J. (2001). *El lenguaje de programación Prolog*. Valencia.

Problema 2: Recursividad duplicada en el ciclo de preguntas

- Descripción detallada: Cuando corríamos el programa en consola, a veces nos repetía la misma pregunta dos veces o se quedaba pegado intentando guardar las respuestas de las profesiones.
- Intentos de solución sin éxito: Nos desesperamos un poco y empezamos a poner cuts (!) por todo el archivo Logic.pl para ver si Prolog dejaba de devolverse. Pero esto empeoró todo, porque el programa dejaba de evaluar otras opciones y terminaba recomendando carreras que nada que ver.
- Soluciones encontradas: Revisando el código con lupa, vimos que teníamos un error de lógica en el If-Then-Else de la regla "preguntar_lista". Estábamos llamando a la recursividad doblemente: una vez adentro de la condición y otra vez afuera. Solo tuvimos que borrar esa última línea que sobraba al final del bloque y el flujo en la consola empezó a funcionar bien.
- Recomendaciones / Conclusiones: Mezclar recursividad con condiciones If-Then-Else en Prolog enreda bastante. Hay que prestar mucha atención para no llamar a la recursividad más de la cuenta y evitar que el programa se vuelva loco haciendo cosas dobles.
- Bibliografía consultada: Apuntes y presentaciones del curso Paradigmas de Programación.

Problema 3: Desfase de preguntas por confusión

- Descripción detallada: Nos pasó algo raro, cuando el usuario escribía algo que el programa no entendía (como "asdfg"), el sistema ponía "No entendí, intenta otra vez" y repetía la misma pregunta. Pero por debajo, la lista de atributos ya había avanzado. Entonces, uno contestaba pensando que estaba respondiendo sobre "tecnología", pero el programa lo estaba guardando en la pregunta que seguía (como "matemáticas"). Se nos hacía un desorden completo con las respuestas.
- Intentos de solución sin éxito: Como venimos acostumbrados a lenguajes como Python o Java, al principio intentamos ver cómo "restarle 1" a un índice o tratar de retroceder un espacio, pero nos topamos con la realidad de que en Prolog las listas no funcionan así y no se puede simplemente "echar para atrás" una vez que saco la cabeza de la lista.
- Soluciones encontradas: Revisando el bloque donde el código avisa que no entendió, nos dimos cuenta de que en la llamada recursiva estábamos mandando solo el Resto de la lista, lo que obligaba al programa a saltarse la pregunta actual. La solución fue facilísima: en el caso de que el "interpretar_texto" falle, le decimos al programa que vuelva a llamar a "preguntar_lista" pero pasándole la lista intacta con todo y su cabeza ([Attr|Resto]). Así el programa se queda "pegado" en la misma pregunta hasta que el usuario responda algo válido.
- Recomendaciones / Conclusiones: Recorrer listas en Prolog requiere mucha concentración en lo que le pasa a la recursividad. Si ocurre un error de validación con el usuario y necesito repetir un paso, tengo que asegurarme de volver a mandar la lista completa ([Cabeza|Cola]) en lugar de mandar solo la cola, para no perder el dato en el que estaba.
- Bibliografía consultada: SWI-Prolog (s.f.). *SWI-Prolog Documentation*. Recuperado de <https://www.swi-prolog.org/>

Conclusiones y Recomendaciones del proyecto

Conclusiones

- Se logró implementar con éxito un sistema experto funcional en Prolog que utiliza gramáticas libres de contexto para interactuar con el usuario en lenguaje natural, cumpliendo con el objetivo de orientar vocacionalmente a los estudiantes.
- La utilización de gramáticas DCG permitió que el sistema fuera flexible, pudiendo interpretar intenciones positivas y negativas sin depender de respuestas cerradas de "sí" o "no", lo cual mejora significativamente la experiencia del usuario.
- El paradigma lógico resultó ser ideal para este tipo de aplicaciones, ya que la separación entre la base del conocimiento (BD.pl) y la lógica de inferencia (Logic.pl) facilita el mantenimiento y la expansión del sistema hacia nuevas carreras en un futuro.

Recomendaciones

- Se recomienda para futuras versiones integrar un módulo de corrección ortográfica más avanzado para evitar que errores de dedo bloqueen el análisis del parser.

- Es aconsejable expandir la base de datos de sinónimos en el archivo de gramática para que el sistema sea capaz de entender una gama aún más amplia de expresiones coloquiales.
- Se sugiere realizar pruebas con usuarios reales fuera del ámbito técnico para identificar frases o estructuras gramaticales que no se hayan contemplado inicialmente.

Bibliografía

- Monferrer, T. E., Toledo, F., & Pacheco, J. (2001). *El lenguaje de programacion Prolog*. Valencia.
- SWI-Prolog (s.f.). *SWI-Prolog Documentation*. Recuperado de <https://www.swi-prolog.org/>
- Instituto Tecnológico de Costa Rica. (2026). *Material del curso Paradigmas de Programación (CE1106)*.